Objects

Named fields, nested records, and copying

Gregory S. DeLozier, PhD

Chapter Route

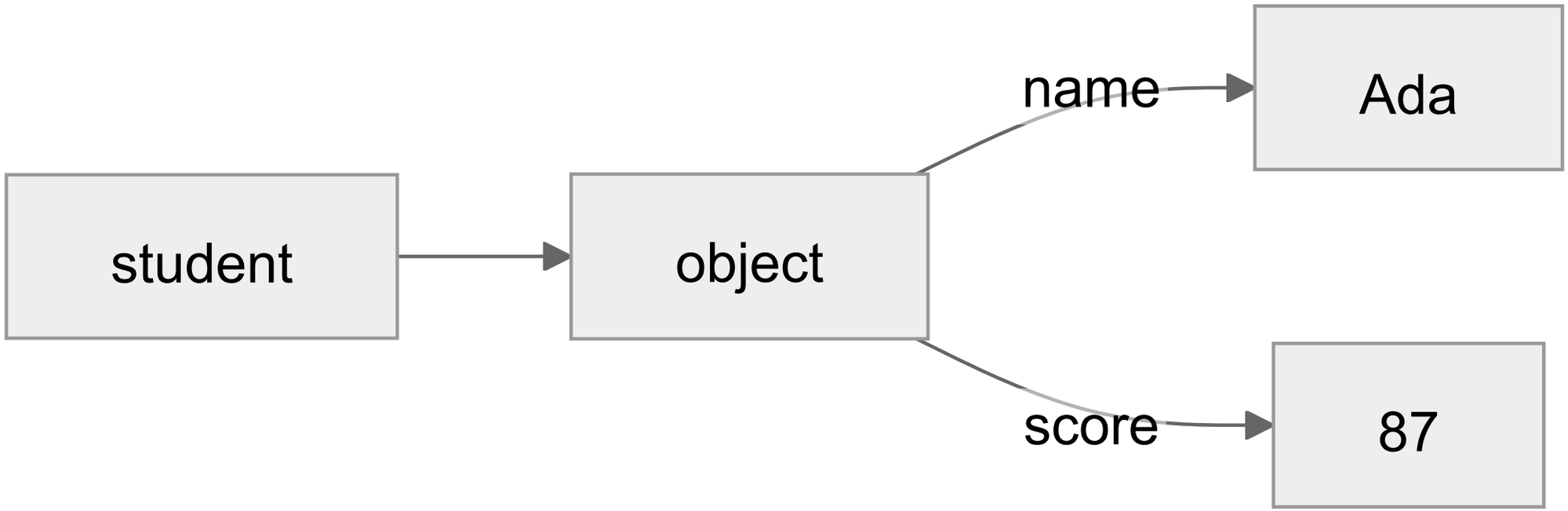
1. Records and named fields
2. Nested records and shared fields
3. Field access in the parser
4. Copying and evaluator rules

Objects in Vertex

- Object literals construct string-keyed records.
- Fields can be read, updated, and added.
- Equality compares keys and values, regardless of insertion order.
- Display preserves field insertion order.

A Record with Named Fields

```
student = {"name": "Ada", "score": 87};
```



Reading, Updating, and Adding

```
student = {"name": "Ada", "score": 87};
print(student.score);
student.score = student.score + 3;
student.passed = true;
print(student.score);
print(student.passed);
```

Prints `87`, then `90`, then `true`.

Equality Ignores Insertion Order

```
a = {"name": "Ada", "score": 87};
b = {"score": 87, "name": "Ada"};
print(a == b);
```

Prints `true`: the same keys have equal values.

Display Preserves Insertion Order

```
print(a);
print(b);
```

Object	Field order in the output
a	name, then score
b	score, then name

Equal objects can have different printed field orders.

Positions and Names

A positional record	A named record
`["Ada", 87]`	`{"name": "Ada", "score": 87}`
`student[0]`	`student["name"]`
`student[1]`	`student.score`

Named fields make each value’s role visible.

Reading a Field

```
student["score"]
student.score

field = "score";
student[field]
```

Brackets evaluate a key. Dot notation spells one.

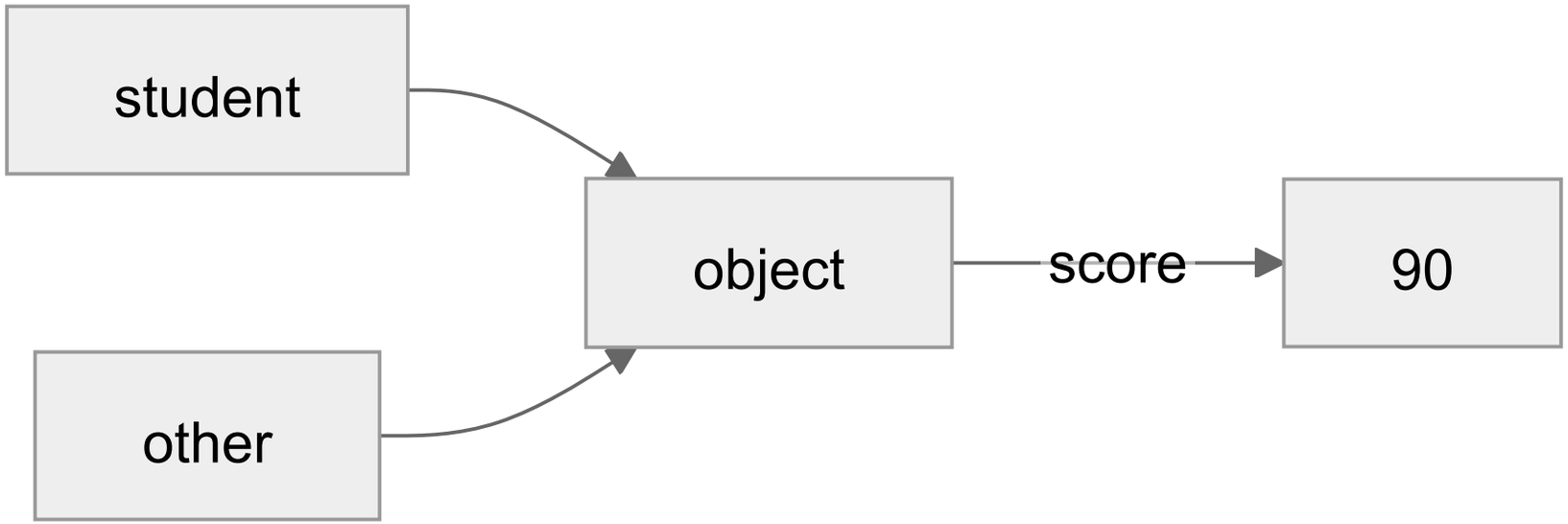
Objects and Lists Together

```
students = [
    {"name": "Ada", "scores": [87, 93]},
    {"name": "Lee", "scores": [82, 90]}
];
students[0].scores[1] = 95;
```

Each suffix selects the next value.

One Record, Two Names

```
student = {"score": 87};
other = student;
other.score = 90;
```



Field Update or New Record

```
student = {"score": 87};
other = student;
other.score = 90;
other = {"name": "Lee", "score": 70};
```

After this statement	student.score	other.score
other = student	87	87
other.score = 90	90	90
other = {"name": "Lee", "score": 70}	90	70

Key Rules

Example	Behavior
`{"na" + "me": "Ada"}`	Computed string key
`{1: "Ada"}`	Key-type error
`{"score": 87, "score": 90}`	Duplicate-key error
`{"score": 87,}`	Syntax error

Objects and Environments

Object field access	Identifier lookup
Starts from a data value	Starts from an environment
Uses a string key	Uses an identifier name
Reads that object directly	May follow environment links
Missing key is an error	Missing binding may be sought outward

A shared host representation does not make them the same thing.

A Field Named $PARENT

```
grandparent = {"name": "Mae", "birthplace": "Kent"};
parent = {"name": "Lee", "$PARENT": grandparent};
print(parent["birthplace"]);
```

Missing-key error. No search through `grandparent`.

Braces Depend on Context

```
student = {"score": 87};

if (student.score >= 60) {
    student.passed = true;
}
```

An expression constructs a value. A block contains statements.

Object Literal Grammar

Extended Backus-Naur Form (EBNF)

```
object_expression ::= "{" [ field { "," field } ] "}"
field             ::= expression ":" expression
```

Syntax permits an expression; evaluation requires a string key.

Dot Notation in the Parser

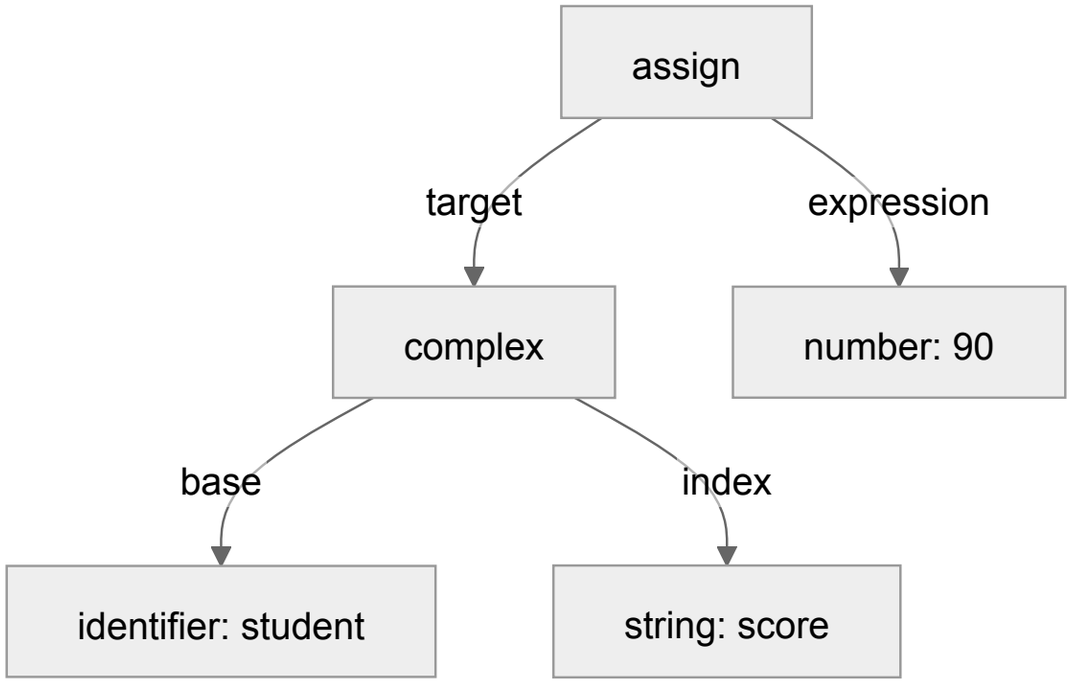
```
student.score
student["score"]
```

```
complex
  base:  identifier("student")
  index: string("score")
```

One abstract syntax tree (AST), two spellings.

Field Assignment Tree

```
student.score = 90;
```



Construction and Status

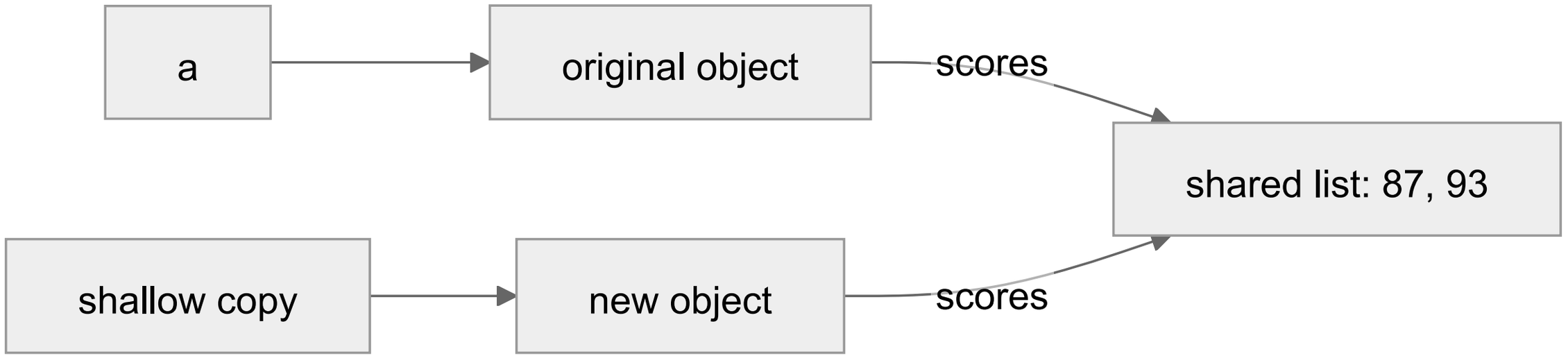
For each field, in source order:

1. Evaluate the key; check status and type.
2. Reject a duplicate key.
3. Evaluate the value; check status.
4. Store the field.

`exit` stops construction before later fields run.

Shallow Copy Shares Nested Values

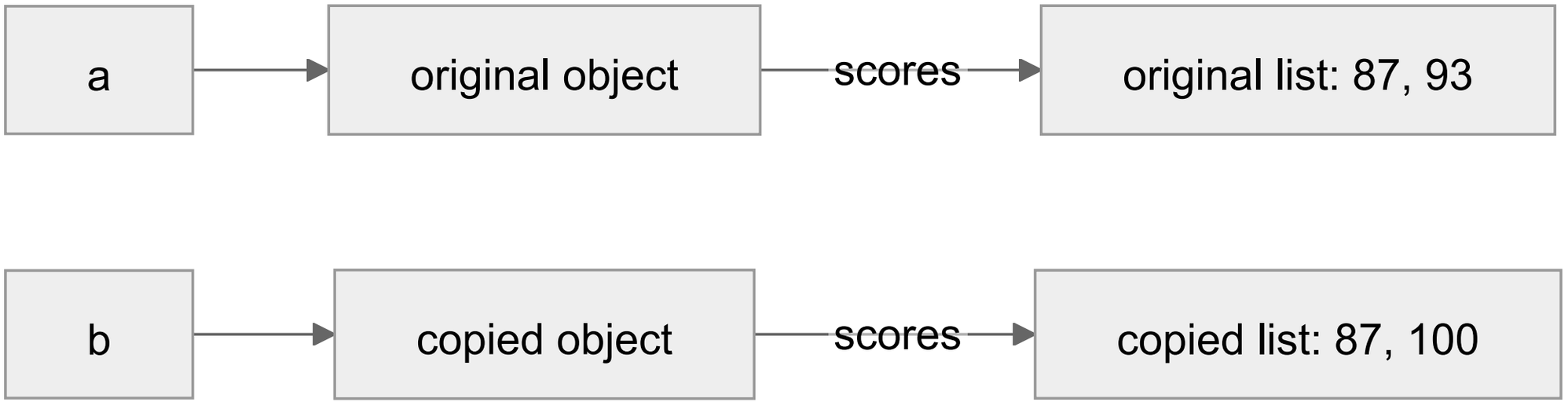
```
a = {"scores": [87, 93]};
```



Conceptual shallow copy: two objects share one list.

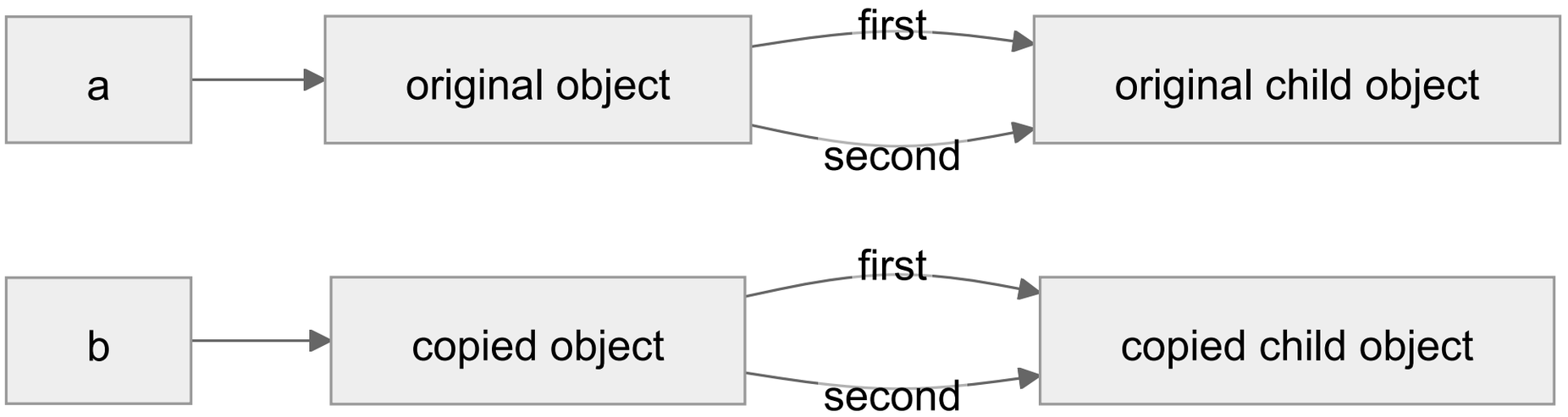
Deep Copy Separates Nested Values

```
a = {"scores": [87, 93]};
b = copy(a);
b.scores[1] = 100;
```



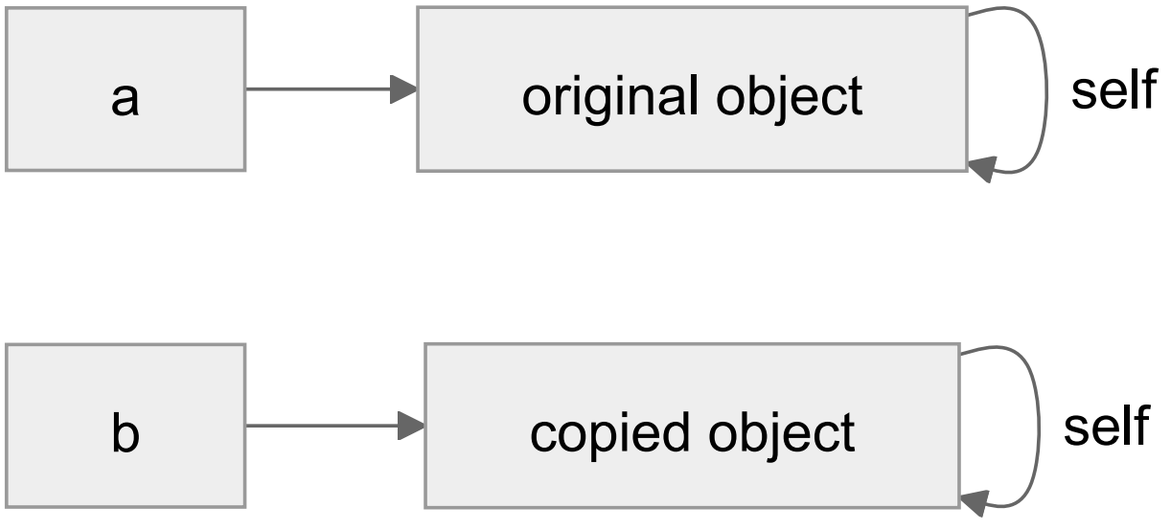
Shared Children Stay Shared

```
child = {"score": 87};
a = {"first": child, "second": child};
b = copy(a);
```



Copying a Cycle

```
a = {};
a.self = a;
b = copy(a);
```



Following the Copied Self-Reference

```
a = {}; a.self = a;
b = copy(a);
b.score = 90;
print(b.self.score);
print(a);
```

Read	Result
`b.self.score`	`90`
`a`	`{"self": {...}}`

The new score belongs to `b`; `a` stays unchanged.

Considerations

New object fields add named facts; list writes use existing positions.

Rebinding one alias leaves the shared object unchanged.

Evaluation checks the base type, key type, and field existence.

Working with Objects

Operation	Result
`type({})`	`"object"`
`string(obj)`	Insertion-order representation
`copy(obj)`	Independent deep copy
`number(obj)`, `boolean(obj)`	Type error

Use fields for named facts; keep field lookup direct.